

```
[HttpPost]
public async Task<ActionResult<Post>> CreatePost(Post post)
{
    await _postsService.CreatePost(post);

    return CreatedAtAction(nameof(GetPost), new { id = post.Id }, post);
}
```

When we call this endpoint, the post object will be serialized in the *JSON* format that is attached to the *POST* request body. In this method, we can get the post from the request and then call the *CreatePost()* method in the service to create a new post. Then, we will return the built-in *CreatedAtAction*, which returns a response message *with the specified action name, route values, and post*. For this case, it will call the *GetPost()* action to return the newly created post.

Note (To Remember for C#): If we don't set the constructor as public then the error msg is:

System.InvalidOperationException: A suitable constructor for type 'MyFirstAPI.Controllers.PostsController' could not be located.

You may find the following terms being used interchangeably: **Dependency Inversion Principle (DIP)**, **Inversion of Control (IoC)**, and **DI**.

IoC is related to **DIP**, but it is not the same. **DIP** concerns decoupling dependencies between high-level modules and low-level modules through shared abstractions (interfaces). **IoC** is used to increase the modularity of the program and make it extensible. There are several technologies to implement **IoC**, such as *Service Locator*, *DI*, the *template method design pattern*, the *strategy design pattern*, and so on.

DI is a form of IoC. This term was coined by Martin Fowler in 2004. It separates the concerns of constructing objects and using them. When an object or a function (**the client**) needs a dependency, it does not know how to construct it. Instead, the client only needs to declare the interfaces of the dependency, and the dependency is injected into the client by external code (**an injector**). It makes it easier to change the implementation of the dependency. It is often similar to **the strategy design pattern**. The difference is that *the strategy pattern* can use different strategies to construct the dependency, while DI typically only uses a single instance of the dependency.

There are three main types of DI:

- **Constructor injection:** The dependencies are provided as parameters of the client's constructor
- **Setter injection:** The client exposes a setter method to accept the dependency
- **Interface injection:** The dependency's interface provides an injector method that will inject the dependency into any client passed to it

DI in ASP.NET Core

ASP.NET Core uses constructor injection to request dependencies. To use it, we need to do the following:

1. Define **interfaces** and their **implementations**.
2. Register the **interfaces** and the **implementations** to the **service container**.
3. Add **services** as the **constructor parameters** to inject the **dependencies**.

DI has four roles: **services**, **clients**, **interfaces**, and **injectors**. In this example, *IPostService* is the interface, *PostService* is the service, *PostsController* is the client, and *builder.Services* is the injector, which is a collection of services for the application to compose. It is sometimes referred to as a DI container.

This DI feature is provided in a NuGet package called [Microsoft.Extensions.DependencyInjection](#). When an ASP.NET Core project is created, this package is added automatically. If you create a console project, you may need to install it manually by using the following command:

`dotnet add package Microsoft.Extensions.DependencyInjection`

In ASP.NET Core, there are three lifetimes when the service is registered:

- **Transient:** A transient service is created each time it is requested and disposed of at the end of the request.
- **Scoped:** In web applications, a scope means a request (connection). A scoped service is created once per client request and disposed of at the end of the request.
- **Singleton:** A singleton service is created the first time it is requested or when providing the implementation instance to the service container. All subsequent requests will use the same instance.

Action injection

Sometimes, one controller may need many services but may not need all of them for all actions. If we inject all the dependencies from the constructor, the constructor method will be large. For this case, we can use action injection to inject dependencies only when needed.

```
[HttpGet]
public ActionResult Get([FromServices] ITransientService
transientService)
{
    ...
}
```

The [[FromServices](#)] attribute enables the service container to inject dependencies when needed without using constructor injection. However, if you find that a service needs a lot of dependencies, it may indicate that the class has too many responsibilities. Based on the *Single Responsibility Principle (SRP)*, consider refactoring the class to split the responsibilities into smaller classes.

Keep in mind that this kind of action injection *only works for actions in the controller*. It does not support normal classes. Additionally, since ASP.NET Core 7.0, the [[FromServices](#)] attribute can be omitted as the framework will automatically attempt to resolve any complex type parameters registered in the DI container.

Keyed services

ASP.NET Core 8.0 introduces a new feature known as *keyed services*, or *named services*. This feature allows developers to register services with a key, allowing them to access the service with that key. This makes it easier to manage multiple services that implement the same interface within an application, as the key can be used to identify and access the service.

We can register them to the service container using different keys:

```
builder.Services.AddKeyedScoped<IDataService,  
SqlDatabaseService>("sqlDatabaseService");  
builder.Services.AddKeyedScoped<IDataService,  
CosmosDatabaseService>("cosmosDatabaseService");
```

Then, we can inject the service by using the `FromKeyedServices` attribute:

```
[ApiController]  
[Route("[controller]")]  
public class KeyedServicesController : ControllerBase  
{  
    [HttpGet("sql")]  
    public ActionResult  
    GetSqlData([FromKeyedServices("sqlDatabaseService")]  
    IDataService dataService) =>  
        Content(dataService.GetData());  
  
    [HttpGet("cosmos")]  
    public ActionResult  
    GetCosmosData([FromKeyedServices("cosmosDatabaseService")]  
    IDataService dataService) =>  
        Content(dataService.GetData());  
}
```

The keyed services can be used to register singleton or transient services as well. Just use the `AddKeyedSingleton()` or `AddKeyedTransient()` method respectively; for example:

```
builder.Services.AddKeyedSingleton<IDataService,
SqlDatabaseService>("sqlDatabaseService");
builder.Services.AddKeyedTransient<IDataService,
CosmosDatabaseService>("cosmosDatabaseService");
```

It is important to note that if an empty string is passed as the key, a default implementation for the service must be registered with a key of an empty string, otherwise the service container will throw an exception.

Using primary constructors to inject dependencies

Beginning with .NET 8 and C# 12, we can use the primary constructor to inject dependencies. A primary constructor allows us to declare the constructor parameters directly in the class declaration, instead of using a separate constructor method.

When using the ***primary constructor in a class***, note that the parameters passed to the class declaration cannot be used as properties or members. For example, if a class declares a parameter named `postService` in the class declaration, *it cannot be accessed as a class member* using `this.postService` or from external code.

Primary constructors can save us from writing fields and constructor methods.

Use a DI container to manage the lifetime of services. If a service implements the `IDisposable` interface, the DI container will dispose of the service when the scope is disposed of. Do not manually dispose of it.

When registering a service, do not use `new` to create an instance of the service. For example, `services.AddSingleton(new ExampleService())`; registers a service instance that is not managed by the service container. So, the DI framework will not be able to dispose of the service automatically.

Avoid using the service locator pattern. If DI can be used, do not use the `GetService()` method to obtain a service instance.

To use *property injection*, *Fun<T> support for lazy initialization*, and so on, you can use third-party DI containers, such as Autofac.

Minimal APIs is a new feature introduced in ASP.NET Core 6.0. It is a new way to create APIs without using controllers. Minimal APIs are designed to be simple and lightweight with minimal dependencies. They are a *good choice for small projects or prototypes, and also for projects that do not need the full features of controllers.*

There is no `Controllers` folder in the project. Instead, you can find the following code in the `Program.cs` file:

```
app.MapGet("/weatherforecast", () =>
{
    var forecast = Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast
        (
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)]
        ))
        .ToArray();
    return forecast;
})
.WithName("GetWeatherForecast")
.WithOpenApi();
```

The preceding code uses the `MapGet()` method to map the GET request to the `/weatherforecast` endpoint. The `MapGet()` method is an extension method of the `IEndpointRouteBuilder` interface. This interface is used to configure the endpoints in the application. Its extension method, `MapGet()`, returns an `IEndpointConventionBuilder` interface that allows us to use fluent APIs to configure the endpoint by using other extension methods, such as `WithName()` and `WithOpenApi()`. The `WithName()` method is used to set the name of the endpoint. The `WithOpenApi()` method is used to generate an OpenAPI document for the endpoint.

ASP.NET Core provides two types of routing: *conventional routing* and *attribute routing*. Conventional routing is typically **used for ASP.NET Core MVC applications**, *while ASP.NET Core web APIs use attribute routing*.

app.MapControllers();

This line of code adds endpoints for controller actions to the *EndpointRouteBuilder instance* without specifying any routes. To specify the routes, we need to use the *[Route]* attribute on the controller class and the action methods.

ASP.NET Core has some built-in route tokens, such as *[controller]*, *[action]*, *[area]*, *[page]*, and so on. These tokens are enclosed in square brackets ([]) and will be replaced with the corresponding values. Note that these tokens are reserved route parameter names and should not be used as a route parameter like {controller}.

ASP.NET Core provides a way to configure the route value globally. Add the following code to the **Program.cs** file: *builder.Services.AddRouting(options => options.LowercaseUrls = true);*

Multiple routes can be applied to one controller class. The following code shows how to apply multiple routes to the `PostsController` class:

```
[ApiController]
[Route("api/[controller]")]
[Route("api/some-posts-whatever")]
public class PostsController : ControllerBase
{
    // Omitted for brevity
}
```

We can add route constraints to route parameters to restrict the values of the route parameters. For example, we can add a route constraint to the `id` parameter to ensure that the `id` parameter is an integer. The following code shows how to add a route constraint to the `id` parameter:

```
[HttpGet("{id:int}")]  
public async Task<ActionResult<Post>> GetPost(int id)  
{  
    // Omitted for brevity  
}
```